

Edition de notes

programmation objets, héritage

On souhaite développer l'application **MyNotes** destinée à éditer et gérer un ensemble de notes (des mémos) qui peuvent contenir du texte et/ou des images. Une note peut par exemple correspondre au compte-rendu d'une réunion ou à des notes prises lors d'une séance de cours.

1 - Définir la classe `Note`

D'un point de vue implémentation, une note est un objet instance de la classe `Note`. Tout objet `Note` est caractérisé par un attribut de type `str` représentant un titre. Le constructeur de cette classe a un paramètre qui permet d'initialiser cet attribut. La classe possède des accesseurs permettant de lire et modifier cet attribut. La méthode `print()` permet d'afficher le titre d'un objet `Note`.

2 - Définir la classe `Article`

Les objets de la classe `Article` sont des objets `Note` qui représentent des notes contenant uniquement du texte.

En plus du titre, les objets `Article` possèdent un attribut de type `str` représentant son texte. La classe possède des accesseurs permettant de lire et modifier cet attribut. Le constructeur de cette classe a deux paramètres qui permettent d'initialiser le titre et le texte d'un article. La méthode `print()` permet d'afficher le titre et le texte d'un objet `Article`.

3 - Définir la classe `Image`

Les objets de la classe `Image` sont des objets `Note` qui représentent des images ou des photos.

En plus d'un titre, les objets `Image` possèdent un attribut de type `str` représentant sa description (qui contient des informations sur le contenu de l'image) et un attribut de type `str` indiquant le chemin du fichier physique de l'image.

La classe possède des accesseurs permettant de lire et modifier ces attributs. Dans l'accesseur en écriture du fichier, on vérifiera que le fichier existe en utilisant la méthode `os.path.isfile()`.

Le constructeur de cette classe a trois paramètres qui permettent d'initialiser le titre, la description et le fichier d'une image.

La méthode `print()` permettra d'afficher le titre, la description, ainsi que son image. On utilisera la librairie `Pil` pour afficher une image. Pour cela, effectuer la commande `pip install Pillow` et dans le code ajouter `import PIL.Image`. La fonction `open()` de ce module ouvre le fichier passé en paramètre et retourne une image. Pour afficher une image on utilise la méthode `show()`.

```
# exemple
import PIL.Image as PILImage

image = PILImage.open(".....")
image.show()
```

4 - Définir la classe Document

On décide aussi de créer la classe `Document` dont les instances sont des objets `Note` qui regroupent virtuellement plusieurs autres objets `Note` afin de constituer un contenu de taille plus importante.

Pour cela, un objet `Document` possède un attribut de type `list` contenant les objets `Note` regroupés par le document.

Le constructeur de cette classe a un paramètre qui permet d'initialiser le titre du document.

Initialement, un objet `Document` ne regroupe aucun objet `Note`. La méthode `ajouter_note()` permet d'ajouter une note de n'importe quel type (`Article`, `Image`, `Document`) au document. La méthode `supprimer_note()` permet de supprimer une note transmise en argument.

La méthode `print()` permet d'afficher l'ensemble des informations d'un objet `Document` c'est-à-dire, son titre suivi de l'affichage des différentes notes qui forment le document.

5 - Main

Écrire le module principal permettant d'entrer différentes notes dans un `Document` et de les visualiser.

6. Modifier la classe Note

attributs et méthodes de classe

On souhaite pouvoir connaître le nombre d'objets `Note` présents dans le système à n'importe quel moment de l'exécution. Ajouter un attribut de classe `_nb_notes` permettant de compter ce nombre d'objets à chaque instant. Définir un accesseur en lecture pour cet attribut. Modifier le constructeur et ajouter un destructeur de manière à ce que l'attribut de classe `_nb_notes` soit toujours à jour.